



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

Answer: The fundamental difference is that the **State Space** represents the physical reality of the environment, while the **Search Tree** represents the paths the agent imagines in its mind.

- **The State Space** is the set of all possible, unique configurations in the world (e.g., the coordinates on the grid). It is finite. A state is just a physical location, like **(2, 3)**.
- **The Search Tree** is a data structure built by the algorithm as it explores. The nodes in a search tree represent *paths* taken to reach a state, not just the state itself. Because there are many different paths to reach the same coordinate, multiple nodes in the search tree can point to the exact same state in the state space. If the agent isn't careful (like in a Tree Search without a **reached** set), the Search Tree can grow infinitely large even if the State Space is very small.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

In graph search theory, the **reached** set solves the problem of **redundant paths and infinite cycles**. By maintaining a memory of previously visited states, the **reached** set prevents the algorithm from revisiting the same coordinate multiple times, effectively upgrading a naive "Tree Search" into a "Graph Search". Without the **reached** set, a DFS (Depth-First Search) agent would be completely blind to its own history. If the agent enters an open area or a cycle in the grid (e.g., being able to move Right, Down, Left, Up in a square), it would continuously push those same coordinates onto its stack. Because DFS aggressively pursues the deepest path first, it would become permanently trapped walking in an infinite loop, constantly expanding an endlessly deep search tree until the program crashes or runs out of memory.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

Visually, **BFS** generates a highly direct and efficient path straight to the food pellet, whereas **DFS** produces a highly erratic, winding, and snake-like path that often explores edges and corners before finally reaching the goal.

Why BFS is optimal: BFS uses a FIFO (First-In, First-Out) queue, which forces it to explore the state space layer-by-layer. It checks all coordinates that are 1 step away, then 2 steps away, then 3 steps away, expanding outward uniformly. Because this specific grid has uniform step costs (every move costs exactly 1), the very first time BFS's "ripple" touches the goal state, it is mathematically guaranteed to be the shortest path.

Why DFS is suboptimal: DFS uses a LIFO (Last-In, First-Out) stack, which forces it to aggressively pursue a single continuous path as deeply as possible until it hits a dead end (a wall or a fully explored corner). It does not care about finding the

shortest route; it only cares about finding the *first* route. Because it blindly plunges down whatever arbitrary branch it chose first, the first path it finds to the goal is usually extremely long, convoluted, and suboptimal.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

The crash in a massive 1000x1000 grid would occur due to severe memory (RAM) bottlenecks, which highlights the stark contrast in space complexity between BFS and DFS.

BFS Memory Bottleneck (Exponential Space): According to the lecture formulas, the space complexity of BFS is $O(b^d)$, where b is the branching factor (up to 4 in our grid) and d is the depth of the shallowest solution. Because BFS explores layer-by-layer, it must keep the entire "perimeter" of the search tree in its queue (frontier) at the same time. On a massive grid, the number of nodes at depth d grows exponentially. BFS and UCS will likely run out of RAM and crash because they are forced to hold tens or hundreds of thousands of unexpanded nodes in memory simultaneously.

DFS Advantage (Linear Space): By contrast, the space complexity of DFS is only $O(bm)$, where m is the maximum depth of the search tree. DFS only needs to store a single continuous path from the start node to the current node, plus the unexpanded sibling nodes along that path. Because it is linear rather than exponential, DFS uses a remarkably small amount of memory. It would comfortably survive a 1000x1000 grid without crashing from memory exhaustion, although the path it eventually finds would be highly suboptimal

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

